CLASE

# Introducción a Ansible



# ¿Qué es Ansible?

### Open source

Creado por Michael DeHaan en 2012. Red Hat lo adquirió en 2015 y lo mantiene junto a una gran comunidad.

### Sin agentes

No instala nada en los servidores: se conecta por SSH (Linux) o WinRM (Windows) y ejecuta las tareas.

### Basado en YAML

Las automatizaciones se escriben en YAML, un formato pensado para personas: se lee casi como una lista de pasos.

# ¿Qué es Ansible?

Una herramienta de automatización IT que configura servidores, despliega aplicaciones y orquesta tareas de forma repetible, escribiendo el estado deseado en archivos de texto.



### Configurar servidores

Instalar paquetes, crear usuarios, editar archivos de configuración.



### Desplegar aplicaciones

Copiar el código, aplicar la configuración y reiniciar servicios en orden.



### Orquestar

Coordinar muchos servidores: primero la base de datos, después la web.

# Beneficios de Ansible

### Simplicidad

YAML legible: cualquiera del equipo entiende qué hace un playbook.

### Sin agentes

Nada que instalar, actualizar ni vigilar en los servidores.

### Idempotencia

Ejecutarlo 1 o 100 veces deja el servidor en el mismo estado.

### Control de versiones

Los playbooks viven en Git: historial, revisión y vuelta atrás.

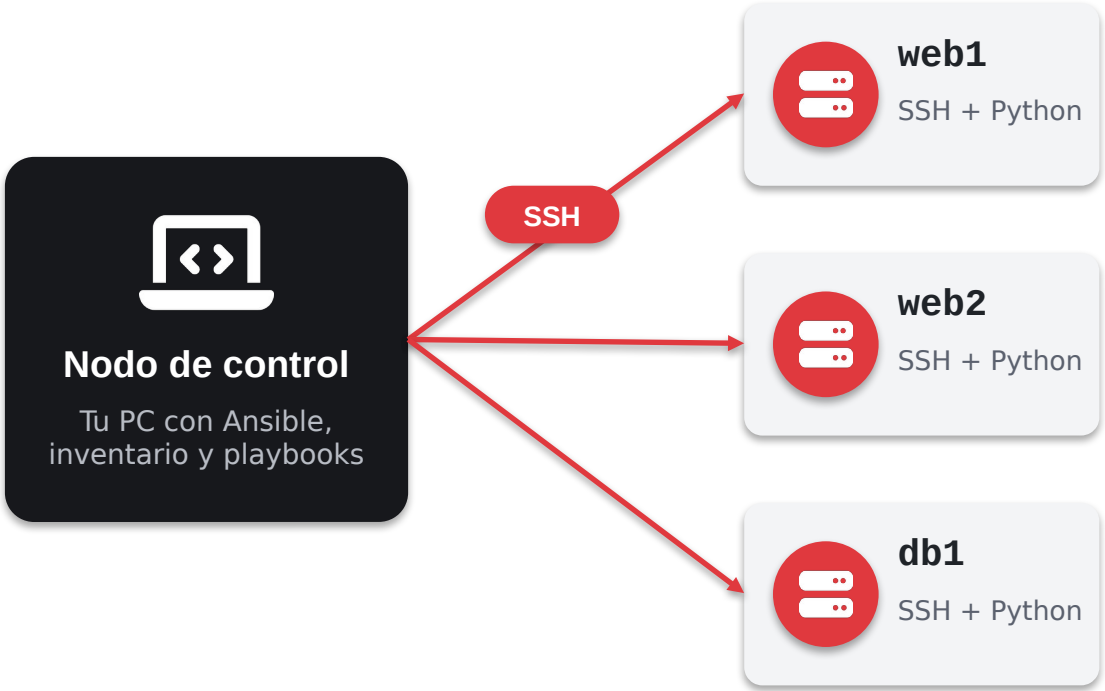
### Colaboración

El equipo revisa cambios de infraestructura como si fuera código.

### Consistencia

Todos los servidores configurados igual, sin diferencias manuales.

# Arquitectura sin agentes




## Nodo de control

- Linux o macOS (Windows: vía WSL)
- Ansible instalado
- Clave SSH

## Nodos gestionados

- Acceso SSH (usuario + clave)
- Python instalado
- ¡Nada más!

*Ansible envía cada tarea, la ejecuta y borra los temporales.*

# Manual vs Ansible

## Proceso manual

- ssh al servidor 1
- sudo dnf install httpd
- Editar la configuración a mano
- Repetir en el servidor 2, 3, 4...
- Lento, con errores y sin registro de qué se hizo

## Con Ansible

- Describes el estado deseado una vez
- Un comando, N servidores
- El playbook queda en Git como documentación

TERMINAL

```
$ ansible-playbook web.yml
PLAY RECAP ***
web1 : ok=6 changed=3
web2 : ok=6 changed=3
```

# Componentes principales

### Nodo de control

La máquina donde instalas y ejecutas Ansible.

### Nodos gestionados

Los servidores que Ansible configura (hosts).

### Inventario

Lista de hosts organizados en grupos: DÓNDE actuar.

### Módulos

Unidades que hacen el trabajo: dnf, copy, user, service...

### Tareas y playbooks

Una tarea llama a un módulo; un playbook agrupa tareas: QUÉ hacer.

### Roles y colecciones

Paquetes reutilizables de tareas, plantillas y módulos.

# Terraform vs Ansible

Herramientas distintas y complementarias

## Terraform · IaC

- Aprovisiona: crea VMs, redes, firewalls
- Guarda estado (.tfstate)
- Crea y destruye (destroy)
- Similares: CloudFormation, Pulumi

## Ansible · Configuración

- Configura lo de dentro: paquetes, archivos
- Sin estado: revisa el servidor cada vez
- Conexión SSH, sin agentes
- Similares: Chef, Puppet, Salt

**Terraform** crea la EC2 → `terraform output` da la IP → **Ansible** la configura

# Declarativo e idempotente

Cada tarea describe el ESTADO que quieres (paquete instalado, servicio iniciado), no los comandos para llegar a él.

- **Idempotencia:** ejecutarlo varias veces da el mismo resultado.
- **1.ª ejecución:** hace los cambios → changed
- **2.ª ejecución:** ya está así → ok, sin cambios
- Si alguien cambia algo a mano, Ansible lo devuelve al estado definido.

TERMINAL

```
# Primera ejecución
TASK [Instalar paquetes extra] ***
changed: [demo]
demo : ok=6 changed=3 failed=0
```

TERMINAL

```
# Segunda ejecución
TASK [Instalar paquetes extra] ***
ok: [demo]
demo : ok=6 changed=0 failed=0
```

# ¿Cómo empiezo con Ansible?

1

## Instalar Ansible

pipx install ansible-core
o el gestor de paquetes (dnf / apt)

2

## Preparar el acceso SSH

Clave SSH y un usuario con sudo en los servidores

3

## Crear el inventario

Lista de servidores y grupos: [web], [db]...

4

## Escribir el primer playbook

Empieza simple: un ping, un paquete, un servicio

# RESUMEN

- Ansible automatiza la configuración de servidores, despliegues y orquestación.
- Es sin agentes: usa SSH y Python en los servidores gestionados.
- Las automatizaciones se escriben en YAML: playbooks legibles por personas.
- Es idempotente: describe el estado deseado y solo cambia lo necesario.
- Componentes: nodo de control, inventario, módulos, tareas, playbooks y roles.
- Complementa a Terraform: Terraform crea la VM y Ansible la configura.

CLASE

# Conceptos básicos de YAML



# YAML: el lenguaje de Ansible

### Formato de datos

No es un lenguaje de programación: representa datos con claves, valores, listas y diccionarios.

### Fácil de leer

Pensado para personas. Sin llaves ni punto y coma: la estructura se ve a simple vista.

### La indentación importa

Los espacios definen qué va dentro de qué. Usa siempre espacios, nunca tabuladores.

# Sintaxis básica de YAML

ejemplo.yml

```yaml
---
# Esto es un comentario
curso: Automatización       # clave: valor
alumnos: 25                 # número
activo: true                # booleano
herramientas:               # lista
  - terraform
  - ansible
servidor:                   # diccionario
  nombre: web1
  puerto: 80
```

| | |
|---|---|
| `---` | Inicio del documento |
| `clave: valor` | Un dato con nombre |
| `- elemento` | Un elemento de una lista |
| `  anidado:` | Diccionario: 2 espacios más |
| `# texto` | Comentario (se ignora) |

# Anatomía de un playbook

web.yml

```yaml
---
- name: Configurar servidor web
  hosts: web
  become: true

  tasks:
    - name: Instalar Apache
      ansible.builtin.dnf:
        name: httpd
        state: present

    - name: Iniciar Apache
      ansible.builtin.service:
        name: httpd
        state: started
```

**Play**

name + hosts: a qué grupo del inventario se aplica.

**become**

Ejecutar como root (equivale a sudo).

**tasks**

Lista de tareas. Se ejecutan en orden, de arriba a abajo.

**Módulo**

ansible.builtin.dnf: la pieza que hace el trabajo.

**Argumentos**

name, state: cómo debe quedar el recurso.

# Anatomía de una tarea

```yaml
- name: Instalar Apache
  ansible.builtin.dnf:
    name: httpd
    state: present
```

→ **Nombre: lo que verás en la salida**

→ **Módulo: colección.nombre (FQCN)**

→ **Argumento: qué paquete**

→ **Argumento: estado deseado**

**Las tareas se ejecutan en orden**

**Cada tarea usa un módulo**

**Toda tarea debería tener name**

# Guía de estilo YAML

- 2 espacios por nivel; nunca tabuladores
- Empieza cada archivo con ---
- Pon name a todas las tareas y plays
- Usa el nombre completo del módulo: ansible.builtin.copy
- Booleanos como true / false
- Comillas si el valor empieza por {{ variable }}
- Extensión .yml en todos los archivos

✗ Mal

mal.yml

```yaml
- dnf: name=httpd
  become: yes
- copy: src=a dest={{ ruta }}
```

✓ Bien

bien.yml

```yaml
- name: Instalar Apache
  ansible.builtin.dnf:
    name: httpd
    dest: "{{ ruta }}"
```

# Buenas prácticas al escribir playbooks

### Extensión .yml

Ansible, los editores y los linters lo reconocen sin problemas.

### Formato consistente

Usa ansible-lint o la extensión de Ansible en VS Code.

### Organización

Variables en group_vars, plantillas en templates/, roles para reutilizar.

### Comentarios y README

Explica el porqué y cómo ejecutar el proyecto.

### No hardcodear valores

Usa variables para lo que cambia: IPs, versiones, usuarios.

# RESUMEN

- YAML es un formato de datos legible; Ansible lo usa para playbooks, inventarios y variables.
- La indentación define la estructura: 2 espacios, nunca tabuladores.
- Estructuras básicas: clave: valor, listas con - y diccionarios anidados.
- Un playbook tiene plays; cada play tiene hosts y una lista de tareas.
- Cada tarea tiene un name y un módulo con sus argumentos.
- Sigue la guía de estilo y valida con ansible-lint.